Site Streak – App Plan & Architecture

Oct 29, 2026 · @Caleb Brown

Overview

Site Streak (working title) is a mobile-first web app that turns daily site reports, QC checks and task handovers into a Duolingo-style game, so reports arrive complete, on time and checked by someone else.

Design principles:

- **Under 5 minutes a day on site.** One thumb, big buttons, works offline on cheap Android phones.
- **Reward quality, not volume.** XP is multiplied by the reviewer's star rating.
- **Every report gets a second pair of eyes.** The level above, or a peer from another site, reviews it.
- **Configurable without code.** Admins set task templates, fields and unlock rules in Settings.
- **Reports drive real outcomes.** They feed progress claims, earned value and a site health score.

Roles and features

Each role has its own recurring tasks, reviews the level below (or a peer), and can assign one-off tasks downward.

Role	Own tasks	Reviews	Assigns to
Foreman	Daily site report, QC hold points, toolbox talk, quantities installed	None	None
Site manager	Approvals, NCRs, progress and materials, weekly summary	Own foremen + 1 foreman report from another site	Foremen
Project manager	Programme vs actual, variations, set team quest	1 site manager summary from another project	Site managers
Construction manager	Portfolio review, monthly targets	PM monthly reports	Project managers

Role	Own tasks	Reviews	Assigns to
Admin	Templates, users, sites, baseline import	None	None

MVP (pilot on 2 sites): sign-in and roles, sites and projects, fixed templates (daily report, hold point), Today screen, offline report with photos, site manager review with stars, XP, streaks, weekly league, Brix mascot with core animations.

Later: Settings template builder, unlock rules, assigned-task cascade with sub-tasks, cross-site reviews with accuracy scoring, team quests, push notifications, BoQ import, earned value, site health, PDF export, reward store.

Learning and training

Line managers assign learning the same way they assign any task: site managers to foremen, PMs to site managers, construction managers to PMs.

- **Micro-lessons:** a 2–5 min video plus a short quiz. No skipping ahead, questions shuffled, XP only on a pass. Videos can be downloaded on Wi-Fi to watch offline.
- **Trainer sessions:** attendees check in by scanning the trainer's QR code, which expires after the session. The trainer then signs each person off as competent or needing more practice.
- **Skill paths:** lessons unlock in order per role and end in a badge such as Hold-Point Inspector. A task can require a badge before it unlocks.
- **Training record:** each person's lessons, sessions and sign-offs, exportable for HR, compliance or client reporting.

Adds four collections: trainingModules, trainingSessions, attendance and competencies.

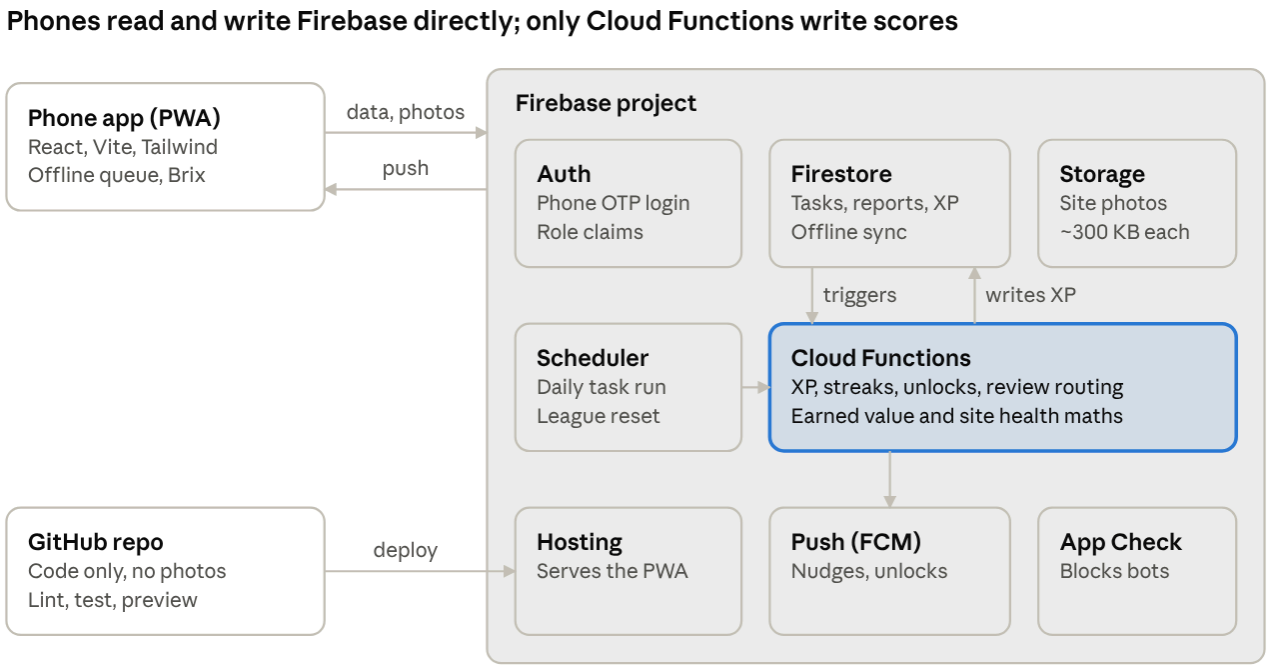
Look and feel

Friendly and playful, built around an original mascot rather than borrowed Duolingo assets.

- **Palette and type:** cream background #FFF7E8, ink #24324A, sunny yellow #FFC93C (primary), teal #2BB3A3 (done), coral #FF7A59 (QC). Nunito font. Chunky rounded buttons with a 4 px bottom shadow that press down on tap.
- **Brix the mascot:** a brick in a yellow hard hat. States: idle bob, cheering (task done), worried (streak at risk), sleeping (day off), surprised (Hawk Eye catch). Start as an SVG animated with Framer Motion; move to Rive later for a proper state machine.

- **Task path:** today's tasks sit on a winding path. Done nodes are teal, the current one pulses, locked ones show who they're waiting on, and a daily chest sits at the end.
- **Moments of delight:** confetti and an XP count-up on submit, a streak flame that grows at 7, 30 and 100 days, a league promotion animation, light haptics.
- **Rewards:** badges, streak freezes, league promotions, plus real rewards the admin sets (airtime, PPE, site lunch).
- **Restraint:** respect reduced-motion settings, sound off by default, no animation slows down a submit.

Architecture



system architecture · phone, Firebase, GitHub

The phone talks to Firebase directly. Scoring, unlocks and review routing run only in Cloud Functions, triggered by Firestore writes or the scheduler. GitHub holds the code; photos go to Firebase Storage, not the repo.

- **Web app:** React + TypeScript + Vite as a PWA, Tailwind, Framer Motion (Rive later for Brix). Wrap with Capacitor later if you need app-store builds.
- **Firebase:** Hosting, Auth (phone code login suits site staff), Firestore with offline persistence, Storage, Functions v2, scheduled functions, FCM push, App Check.
- **GitHub:** monorepo with `apps/web`, `functions` and `packages/shared`. Actions run lint and tests on each PR, deploy a preview channel, and deploy live on merge to main.
- **Local dev:** Firebase Emulator Suite for everything, including security-rule tests.

Data model and task engine

All data sits under `orgs/{orgId}/` so several companies can share one Firebase project. Roles and orgId live in Auth custom claims.

Collection	Holds	Key fields
users	One doc per person	role, siteIds, xpTotal, streak, streakFreezes, league, reviewAccuracy
projects, sites	Where work happens	name, timezone, members, baselineId
taskTemplates	Recurring task definitions (Settings)	role, frequency, dueTime, xp, fields[], reviewRule, unlockRules[]
tasks	Every task instance, recurring or assigned	templateId, assigneeId, assignedById, siteId, dueAt, status, dependsOn[], parentTaskId
submissions	What was filled in	taskId, answers, photos[], gps, capturedAt, syncedAt
reviews	Ratings and checks	submissionId, reviewerId, stars, checks, comment, crossSite, auditedBy
xpEvents	Append-only XP ledger	userId, source, amount, multiplier, createdAt
leagues/{weekId}/members	Weekly league tables	userId, tier, weeklyXp, rank
quests	Team goals	scope, target, progress, reward, endsAt
boqItems, progressEntries	Earned value (later)	unit, qty, rate, plannedDates / qtyInstalled, verifiedBy

How the engine runs:

1. A scheduled function creates each person's task instances from templates at midnight site time.
2. A task starts `locked` if its unlock rules aren't met. When any task completes, a function re-checks its dependents, unlocks them and notifies the assignee.

3. Assigned tasks can only go to a lower role in the same project. XP is capped by role, and a parent task completes when all its sub-tasks do.
4. On submit, a function creates review tasks: the own site manager, one random cross-site reviewer, and a 10% audit sample for the level above.
5. XP, streaks and leagues are calculated server-side only and written to the xpEvents ledger. The app never writes XP itself.

Task status runs `locked → open → submitted → reviewed`, with `late` and `excused` as side states.

Progress and earned value

Foremen log quantities, never money; the app turns verified quantities into earned value against an imported baseline.

1. **Baseline (PM or QS, once per project):** import the BoQ as CSV from the IPC workbook: item, unit, quantity, rate, planned start and finish. This gives budget and planned value.
2. **Daily (foreman):** log quantity installed against BoQ items in the site report, such as m² of slab or number of trusses.
3. **Weekly (site manager):** verify the quantities as a review task. Only verified quantities count.
4. **Calculated:** earned value = verified quantity × rate. SPI = earned value ÷ planned value. CPI only if actual costs are imported from cost or LPO records.
5. **Monthly:** compare app quantities with the QS measure for the IPC. Close agreement earns a Measurement Accuracy badge; a gap over 10% raises a flag.

Site health score = report completeness + first-time pass rate + open NCRs + SPI, weighted in Settings. Rules can create tasks, for example SPI below 0.9 creates "Submit a recovery plan" for the PM.

Security, offline and anti-gaming

The game only works if the points can't be faked, so scoring lives on the server and evidence is checked.

- **Security rules:** users read only their own org; write only their own submissions; review only when assigned. XP, streaks, unlocks and review routing are written by Cloud Functions alone. App Check blocks scripted clients.
- **Offline:** Firestore offline persistence; photos queue on the phone and upload on reconnect; photos compressed to about 300 KB. Capture time comes from the device, and a sync more than 24 hrs late is flagged.

- **Evidence:** photos only from the in-app camera with GPS and time; duplicate photo check by image hash; geofence per site.
- **Fair play:** reviewer accuracy audits, occasional planted errors for Hawk Eye, a daily XP cap, and no reward for "zero incidents" (reward reporting instead).
- **Data care:** photos may show people, so set a retention period per org and keep access role-based.

Build phases

Ship a small MVP to two sites first and only build settings and earned value once people are actually using it. Durations are rough estimates.

1. **Phase 0, setup (about 1 week):** GitHub repo, Firebase project, emulators, CI deploys, sign-in with roles, seed data.
2. **Phase 1, MVP (4–6 weeks):** fixed templates, Today screen, offline report with photos, site manager review, XP, streaks, weekly league, Brix and core animations. Pilot on 2 sites.
3. **Gate:** at least 90% of daily reports on time for 4 weeks of the pilot before moving on.
4. **Phase 2, workflow (about 4 weeks):** Settings template builder, unlock rules, assigned-task cascade, learning paths with trainer sign-off, cross-site reviews with accuracy, team quests, push notifications.
5. **Phase 3, progress (4–6 weeks):** BoQ import, quantities, earned value and SPI, site health, PDF export, reward store.

Claude Code kickoff prompt

Paste this into Claude Code in an empty folder linked to your GitHub repo. It builds Phase 0 and the first slice of Phase 1, pausing after each step.

```
I'm building "Site Streak", a gamified construction site-reporting PWA (think
Duolingo for daily site reports and QC). Set up the project and build Phase 0
plus the first slice of Phase 1.

STACK (don't substitute)
- React 18 + TypeScript + Vite, Tailwind CSS, Framer Motion, vite-plugin-pwa
- Firebase: Hosting, Auth (email link + phone OTP), Firestore (offline
persistence on), Cloud Storage for photos, Cloud Functions v2 (TypeScript,
Node 20), scheduled functions, FCM
- Firebase Emulator Suite for all local dev and tests
- GitHub repo with GitHub Actions: lint + test on PR, deploy a preview
channel on PR, deploy live on merge to main
```

```
REPO LAYOUT (npm workspaces)
/apps/web  /functions  /packages/shared (types, zod schemas, XP rules)
firestore.rules  storage.rules  /docs

DATA MODEL
Multi-tenant under orgs/{orgId}/: users, projects, sites, taskTemplates,
tasks, submissions, reviews, xpEvents (append-only ledger),
leagues/{weekId}/members, quests. Roles via Auth custom claims: foreman,
siteManager, projectManager, constructionManager, admin.

RULES THAT MATTER
- XP, streaks, task unlocks and review assignment are written ONLY by Cloud
Functions. Clients can never write XP fields or xpEvents. Prove it with
security-rule tests in the emulator.
- XP = base XP x reviewer star multiplier (1 star x0.5, 2 stars x1, 3 stars
x1.5), with a daily cap per user.
- A streak breaks if the daily report isn't submitted by the template's due
time in the site's timezone. Streak freezes cover rain days and holidays.
- Reports must work offline: queue submissions and photos, sync later.
Compress photos on the phone to about 300 KB. Capture GPS and time from the
in-app camera.

BUILD IN THIS ORDER - stop after each step, summarise, and wait for me
1. Scaffold the monorepo, Firebase config, emulators, Tailwind theme and CI
workflows.
2. Auth, role claims and a seed script: 1 org, 2 sites, 1 user per role, plus
"Daily site report" and "Rebar hold point" templates.
3. Shared zod schemas and TypeScript types for every collection.
4. Firestore and Storage security rules with emulator tests.
5. Foreman "Today" screen: task path (done / current / locked / daily chest),
streak, XP and league chips, bottom nav.
6. Daily site report form generated from the template's fields (pick list,
number, text, photo), working offline.
7. Functions: onSubmission creates a review task for the site manager;
onReview writes an xpEvent and updates totals; a daily scheduled job
generates task instances and checks streaks.

LOOK AND FEEL
Friendly and playful, not corporate. Cream background #FFF7E8, ink #24324A,
sunny yellow #FFC93C (primary), teal #2BB3A3 (done), coral #FF7A59 (QC).
Nunito font. Chunky rounded buttons with a 4px darker bottom shadow that
press down on tap. Mascot "Brix": a brick wearing a yellow hard hat, built as
an SVG React component with idle, cheer and worried states animated in Framer
Motion. Confetti and an XP count-up on submit. Respect prefers-reduced-
motion. Don't copy Duolingo's owl, colours or assets.
```

```
CONSTRAINTS
Keep it simple; no over-engineering. Mobile-first at 390px, touch targets at least 44px, fast on cheap Android phones over patchy 3G. Code lives in GitHub; photos go to Firebase Storage, never the repo. Write a short README with setup steps. Ask me before adding any library not listed here.
```